Docker Containerization Guide
IBM Telco Customer Churn Prediction

Machine Learning Operations

February 9, 2026

Contents

1 Introduction to Docker

Docker is a containerization platform that packages an application with all its dependencies into a single standardized unit called a container.

1.1 Why Docker?

- Reproducibility: Same environment everywhere (dev, test, production)
- Isolation: No conflicts with system packages or other projects
- Portability: Run on any machine with Docker installed
- Scalability: Easy to deploy multiple instances
- Simplicity: One command to run the entire application

1.2 Docker vs Virtual Machine

Docker containers are lightweight compared to virtual machines. A typical Docker container is 100 MB to 1 GB, while a VM is 2-20 GB. Containers start in seconds; VMs take minutes.

2 Project Structure

Project files:

- Dockerfile - Docker configuration
- api.R - Plumber REST API
- TRAIN_IBM_TELCO_85.R - Model training script
- xgboost_churn_model.rds - Pre-trained model
- WA_Fn-UseC_-Telco-Customer-Churn.csv - Training data

3 Understanding the Dockerfile

The Dockerfile contains instructions to build a Docker image.

3.1 Step 1: Base Image

```
FROM rocker/r-ver:4.3.2
```

Start from the rocker/r-ver:4.3.2 image, which has R 4.3.2 pre-installed.

3.2 Step 2: Install System Dependencies

```
RUN apt-get update && apt-get install -y \
    libcurl4-openssl-dev \
    libssl-dev \
    libxml2-dev \
    libsodium-dev
```

Install system libraries required by R packages:

- libcurl4-openssl-dev: For HTTP requests (Plumber needs this)
- libssl-dev: SSL/TLS encryption
- libxml2-dev: XML parsing
- libsodium-dev: Cryptography

3.3 Step 3: Install R Packages

```
RUN R -e "install.packages(c('plumber', 'xgboost'), \
    repos='http://cran.us.r-project.org')"
```

Install R packages:

- plumber: Framework for building REST APIs in R
- xgboost: Machine learning library for predictions

3.4 Step 4: Setup Working Directory

```
WORKDIR /app
COPY api.R .
COPY xgboost_churn_model.rds .
```

Set working directory to /app and copy application files.

3.5 Step 5: Expose Port

```
EXPOSE 8000
```

Document that the application listens on port 8000. Use -p 8000:8000 when running.

3.6 Step 6: Start Command

```
CMD ["R", "-e", "plumber::plumb('api.R')$run(port=8000)"]
```

Default command executed when container starts: Load and run the Plumber API on port 8000.

4 Building the Docker Image

4.1 What is an Image?

A Docker image is a blueprint containing the operating system, packages, code, and configuration.

4.2 Build Command

```
docker build -t telco-churn:latest .
```

Parameters:

- docker build: Build a new image
- -t telco-churn:latest: Tag with name and version
- .: Use Dockerfile from current directory

4.3 Build Process

1. Docker reads the Dockerfile
2. Executes each instruction sequentially
3. Creates intermediate images for each layer
4. Final image is ready to run

Expected build time: 5-15 minutes (first build is slower).

5 Running the Docker Container

5.1 What is a Container?

A container is a running instance of an image. Think of it as a recipe (image) becoming a cooked meal (container).

5.2 Run Command

```
docker run -p 8000:8000 \
    --name telco-api \
    telco-churn:latest
```

Parameters:

- docker run: Create and start a container
- -p 8000:8000: Map port 8000 from container to host
- –name telco-api: Name this container
- telco-churn:latest: Image to use

6 Docker Commands Reference

6.1 Image Commands

```
docker images                      # List all images
docker rmi telco-churn:latest      # Remove image
docker inspect telco-churn         # View image details
docker history telco-churn         # View build history
```

6.2 Container Commands

```
docker ps                          # List running containers
docker ps -a                       # List all containers
docker stop telco-api              # Stop container
docker start telco-api             # Start stopped container
docker logs telco-api              # View container logs
docker logs -f telco-api           # Follow logs live
docker rm telco-api                # Remove container
```

7 API Usage

7.1 Health Check

```
curl http://localhost:8000/health
```

Response:

```
{
  "status": "ok",
  "model": "XGBoost Telco Churn",
  "accuracy": 0.85,
  "auc": 0.92
}
```

7.2 Make Prediction

```
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{
    "gender": 0,
    "tenure": 24,
    "MonthlyCharges": 65.5,
    "TotalCharges": 1575.0
  }'
```

8 Comparison: Docker vs Direct Execution

Aspect	Direct	Docker
Setup Time	Quick (2-5 min)	Slow first (10-15 min)
Dependencies	Possible issues	None guaranteed
Reproducibility	Difficult	Guaranteed
Deployment	Manual	Automated
Portability	Limited	Excellent
Resource Use	Efficient	Lightweight

9 When to Use Docker

9.1 Use Docker When

- Deploying to production servers
- Sharing code with team members
- Running on different machines (Windows, Mac, Linux)
- Setting up CI/CD pipelines
- Scaling with multiple instances

9.2 Skip Docker When

- Quick prototyping on local machine
- One-time analysis or research
- Limited computational resources
- Not sharing code

10 Quick Start Guide

10.1 Option 1: Without Docker (Fastest)

```
cd /home/prajwa/-telco-churn-xgboost-shap-
Rscript -e "plumber::plumb('api.R')\$run(port=8000)"
```

Then test:

```
curl http://localhost:8000/health
```

10.2 Option 2: With Docker

```
cd /home/prajwa/-telco-churn-xgboost-shap-
docker build -t telco-churn .
docker run -p 8000:8000 telco-churn
```

Then test in another terminal:

```
curl http://localhost:8000/health
```

11 Troubleshooting

11.1 Container Won't Start

Check logs:

```
docker logs telco-api
```

Or run with interactive shell:

```
docker run -it telco-churn:latest bash
```

11.2 Port Already in Use

Use a different port:

```
docker run -p 9000:8000 telco-churn:latest
```

12 Conclusion

Docker provides a standardized, reproducible way to package and deploy the Telco Churn prediction API. While not strictly necessary for development, it is essential for production deployments, team collaboration, and consistent environments across machines.

The choice between Docker and direct execution depends on your use case. For development and testing, running R scripts directly is faster. For production and sharing, Docker is the better choice.

Docker makes your application: Portable, Reproducible, and Production-Ready